
TP 3 - SIMULACIÓN DE UN MODELO MM1 E INVENTARIO

Alvaro Tomasino

Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
atomasino08@gmail.com

Juan Bautista Martinelli

Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
martinellij4@gmail.com

Tomás Aguilera

Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
tomas.aguilera.27@gmail.com

Melina Aronson

Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
melinaaronson@gmail.com

3 de julio de 2026

ABSTRACT

El objetivo de este informe es medir qué tan bien funcionan el sistema de colas M/M/1 y el modelo de inventario bajo distintas situaciones, comparando los cálculos matemáticos con programas en Python y modelos en AnyLogic.

Keywords Simulación · Modelo MM1 · Modelo de Inventario · AnyLogic

1. Introducción

En cualquier negocio, manejar bien las filas de espera y el stock de productos es fundamental para no perder dinero ni tiempo. La teoría de colas nos da herramientas para entender sistemas donde la gente llega y espera ser atendida. El modelo M/M/1 permite predecir, por ejemplo, cuánto tiempo va a estar un cliente en una fila antes de irse.

Por otro lado, los modelos de inventario sirven para controlar el stock, buscando un equilibrio entre el costo de pedir mercadería, mantenerla guardada y el problema de quedarse sin nada para vender.

Aunque existen fórmulas para resolver estos problemas, la simulación computacional es mejor para ver cómo se comportan estos sistemas en el tiempo y con cambios inesperados. En este informe, vamos a comparar los resultados teóricos con los obtenidos en Python y AnyLogic. La idea es probar distintos escenarios para ver si las herramientas de simulación coinciden con la matemática.

2. Marco teórico y fórmulas

2.1. Modelo M/M/1

El modelo M/M/1 representa un sistema de colas con un único servidor, donde los tiempos entre arribos de los clientes y los tiempos de servicios se modelan como variables aleatorias independientes de distribución exponencial. La letra M hace referencia a la propiedad "Markoviana." de falta de memoria (*memoryless*) de la distribución exponencial, que significa que el estado futuro del sistema solo depende del estado actual y no de su historia pasada.

Este sistema se define por tres componentes: proceso de llegada (*arrival process*), mecanismo de servicio (*service mechanism*) y disciplina de la cola (*queue discipline*). Se asume que los clientes llegan uno a la vez y, si encuentran

al servidor ocupado, se incorporan a una única cola que suele seguir una disciplina FIFO (first-in, first-out), donde el primer cliente en llegar es el primero en ser atendido.

2.1.1. Parámetros del sistema

Tasa de arribos (*arrival rate*): Es el recíproco del tiempo promedio entre llegadas ($E(A)$: *mean interarrival time*).

$$\lambda = 1/E(A) \quad (1)$$

Tasa de servicio (*service rate*): Es el recíproco del tiempo de servicio promedio por cliente ($E(S)$: *mean service time*).

$$\omega = 1/E(S) \quad (2)$$

Factor de utilización (*utilization factor*): Representa qué tan cargado está el servidor y para un sistema de un solo servidor se calcula como:

$$\rho = \lambda/\omega \quad (3)$$

2.1.2. Condición de estabilidad y estado estacionario

Un aspecto crítico de la simulación es determinar si el sistema alcanza un estado estacionario, es decir, un punto en el que las características estadísticas del modelo se vuelven constantes tras un periodo de funcionamiento prolongado. Para que estas medidas de rendimiento existan y sean finitas en un modelo M/M/1, se tiene que cumplir la condición de estabilidad $\rho < 1$. Si no se cumple, la cola crecería indefinidamente.

También incorporamos el análisis del **modelo M/M/1/K (cola finita)**, que tiene una capacidad de sistema limitada a k clientes (servidor + cola). A diferencia del sistema infinito, este modelo siempre alcanza un estado estable, aunque $\lambda \geq \omega$, ya que los clientes excedentes son rechazados por el mecanismo de denegación de servicio. Este rechazo impide que la cola crezca indefinidamente, estabilizando el flujo de salida al nivel de la tasa de servicio.

2.1.3. Medidas de rendimiento

Las medidas de rendimiento que se esperan obtener son:

- **L:** Número promedio de clientes en el sistema (incluyendo al que está en servicio).
- **Q:** Número promedio de clientes en la cola.
- **w:** Tiempo promedio de espera en el sistema (espera en cola más tiempo de servicio).
- **d:** Retraso promedio en la cola.

Estas medidas se relacionan por las ecuaciones de conservación (*conservation equations*):

$$Q = \lambda d \quad y \quad L = \lambda w \quad (4)$$

En el caso del modelo M/M/1 en estado estacionario, el número promedio de clientes en el sistema se puede calcular directamente como $L = \rho/(1 - \rho)$. El tiempo total en el sistema se vincula con el retraso en la cola por la expresión $w = d + E(S)$.

También se consideran las medidas probabilísticas:

- **Probabilidad de encontrar n clientes en cola (P_n):** Para un sistema M/M/1 infinito, se define por la expresión analítica $P_n = (1 - \rho)\rho^n$.
- **Probabilidad de denegación de servicio (Bloqueo):** Se calcula como P_k , donde k representa la capacidad máxima del sistema. Esta probabilidad indica la fracción de tiempo que el sistema permanece lleno, y por lo tanto la probabilidad de que un cliente que llega sea rechazado. En este informe, se analiza este comportamiento variando el tamaño de la cola en 0, 2, 5, 10 y 50 lugares.

2.2. Modelo de inventario

El estudio de los sistemas de inventario tiene como objetivo determinar políticas de pedido óptimas para un producto específico, buscando satisfacer la demanda de los clientes mientras se minimizan los costos totales de operación. Estos sistemas se modelan como procesos estocásticos y dinámicos, donde el nivel de inventario cambia instantáneamente ante la ocurrencia de eventos discretos, como la llegada de una demanda o la recepción de un pedido.

2.2.1. Política estacionaria (s, S) (*stationary policy*)

El nivel de inventario se revisa en intervalos de tiempo fijos. Si el nivel de inventario actual (I) es menor al punto de reorden (s), se emite un pedido por una cantidad $Z = S - I$ para alcanzar el nivel de objetivo S . Si el inventario es mayor o igual a s , no se hace ningún pedido en ese periodo.

2.2.2. Componentes del costo

Para evaluar la eficiencia de una política de inventario, las categorías de costos que conforman el costo total (*total cost*) son:

- **Costo de pedido (*ordering cost*):** Es el gasto incurrido al solicitar suministros. Se compone de un costo fijo de preparación o configuración (K) y un costo incremental proporcional a la cantidad de unidades solicitadas ($i \cdot z$).
- **Costo de mantenimiento (*holding cost*):** Representa el costo de capital, almacenamiento y seguros por mantener productos en stock.
- **Costo de faltante (*shortage cost*):** Es la penalización económica por no poder satisfacer la demanda de un cliente, lo que suele traducirse en ventas perdidas o pedidos pendientes (*backlog*).

2.2.3. Tipos de eventos y variables de estado

El sistema se rige por la interacción de 4 tipos de eventos: llegada de un pedido del proveedor, demanda de un cliente, fin de la simulación y evaluación periódica del inventario.

Event description	Event type
Arrival of an order to the company from the supplier	1
Demand for the product from a customer	2
End of the simulation after n months	3
Inventory evaluation (and possible ordering) at the beginning of a month	4

Las variables de estado son: el nivel de inventario ($I(t)$: *inventory level*), la cantidad de pedidos pendientes de entrega (*the amount of an outstanding order from the company to the supplier*) y el tiempo del último evento (*the time of the last event*).

Un factor clave en la dinámica es el retraso de entrega (*delivery lag* o *lead time*), que es el tiempo aleatorio que transcurre desde que se emite una orden hasta que los productos ingresan efectivamente al inventario.

2.2.4. Objetivo de la simulación

Como los componentes del costo suelen reaccionar de forma opuesta ante cambios en los parámetros s y S , la simulación permite estimar el costo total promedio esperado (*expected average total cost*) para distintas configuraciones, y así identificar la política más económica.

3. Metodología

3.1. Python

3.1.1. Modelo de Colas MM1

En este modelo de simulación de colas M/M/1 y M/M/1/K, el comportamiento del sistema puede modificarse mediante distintos parámetros de entrada. La variación de estos parámetros permite representar diferentes escenarios y analizar cómo afectan al rendimiento del sistema, tales como la congestión de la cola, los tiempos de espera, la utilización del servidor y la probabilidad de rechazo de clientes.

3.1.1.1 Definir Parámetros de Entrada En este modelo de simulación de colas M/M/1 y M/M/1/K, el comportamiento del sistema puede modificarse mediante distintos parámetros de entrada. La variación de estos parámetros permite representar diferentes escenarios. Los parámetros que se pueden modificar son los siguientes:

- El tiempo promedio entre arribos (*MEAN_INTERARRIVAL*): $1/\lambda$
- El tiempo promedio de servicio (*MEAN_SERVICE*): $1/\mu$
- Cantidad de clientes a procesar (*NUM_DELAYS_REQUIRED*)
- Tamaño máximo de la cola (*QUEUE_LIMIT*)

3.1.1.2 Generar variables aleatorias En MM1, las distribuciones suelen ser exponenciales, por lo que es importante generar aleatoriedad. Para esto, vamos a usar la biblioteca estándar de Python *random*. Los valores aleatorios del sistema se generan exclusivamente mediante la función:

```
random.expovariate(1.0 / mean)
```

Esta función produce variables aleatorias con distribución exponencial, que en este modelo representan el Tiempo entre arribos de clientes (*MEAN_INTERARRIVAL*) y el Tiempo de servicio del servidor (*MEAN_SERVICE*). Ambos procesos se modelan como variables independientes.

En este código, la semilla se define como:

```
RANDOM_SEED = datetime.datetime.now().microsecond
```

El uso de una semilla dinámica transforma la simulación en un experimento donde se puede Analizar variabilidad de resultados, Evaluar estabilidad estadística de las métricas y Detectar sensibilidad del sistema al azar.

3.1.1.3 Modelar eventos Los eventos se modelaron mediante el enfoque clásico de simulación de eventos discretos. Es decir, el reloj de simulación no avanza continuamente, sino que salta directamente al instante del próximo evento.

Solo hay dos tipos de eventos representados en el código:

Evento	Código	Significado
Arribo	1	Llega un cliente al sistema
Salida	2	Termina el servicio de un cliente

En el código:

```
'time_next_event': {
    1: expon(mean_interarrival), # próximo arribo
    2: float('inf')             # próxima salida
}
```

La próxima salida tiene tiempo infinito por default ya que si no hay clientes que se están atendiendo en el servidor, no hay un tiempo de salida definido. Cada evento tiene asociado un instante futuro. En nuestro código se calcula así:

```
state['time_next_event'][1] = state['sim_time'] + expon(mean_interarrival)
```

Esto significa que el próximo arribo ocurrirá dentro de un tiempo exponencial. Si el reloj actual vale 10 min y se genera un interarribo de 0.7 min:

- $t = 10$
- $\text{interarribo} = 0.7$
- $\Rightarrow \text{próximo arribo} = 10.7$

La selección del siguiente evento se realiza mediante la función *timing(state)*. Donde, entre otras cosas, esta busca el evento más cercano en el tiempo:

```
for event_type, event_time in state['time_next_event'].items():
    if event_time < min_time:
        min_time = event_time
        next_event_type = event_type
```

Para modelar los arribos se programó la función *arrive(...)*. Al ocurrir un arribo se agenda el próximo arribo y se verifica el estado del servidor. Si el servidor está libre:

```
if state['server_status'] == IDLE
```

El cliente entra directamente al servicio y se programa su salida:

```
state['time_next_event'][2] = state['sim_time'] + service_time
```

Si el servidor está ocupado:

```
if state['server_status'] == BUSY
```

si hay espacio en cola, se agrega a la cola. Si la cola está llena, se bloquea la llegada.

Para modelar las salidas se programó la función *depart(...)*. Cuando termina un servicio, si no hay clientes esperando (cola vacía) el servidor pasa a estado ocioso y se elimina el próximo evento de salida (se programa la próxima salida para que sea en tiempo infinito):

```
# Verifica si la cola está vacía
if state['num_in_q'] == 0:
    # La cola está vacía, el servidor pasa a estar inactivo y se desprograma la salida
    state['server_status'] = IDLE
    state['time_next_event'][2] = float('inf')
```

Si hay clientes esperando (cola no vacía) Se toma el primer cliente, se calcula cuánto esperó, se genera un nuevo tiempo de servicio y se agenda la próxima salida:

```
else:
    # La cola no está vacía, se saca al primer cliente de la cola
    state['num_in_q'] -= 1
    arrival_time = state['time_arrival'].pop(0)
    # Calcula el retraso de este cliente y acumula
    delay = state['sim_time'] - arrival_time
    state['total_of_delays'] += delay
    state['num_custs_delayed'] += 1
    # Genera el tiempo de servicio y programa salida
    service_time = expon(mean_service)
    state['time_next_event'][2] = state['sim_time'] + service_time
    # Acumula tiempo en el sistema (retraso en cola + tiempo de servicio)
    state['total_time_in_system'] += delay + service_time
```

3.1.1.4 Métricas a evaluar A partir de esto vamos a ir recolectando las métricas de las 10 corridas exigidas y con el barrido de parámetros correspondiente. Las métricas basadas en promedios temporales utilizan el método de áreas. Antes de procesar cada evento, la función *update_time_avg_stats()* calcula el tiempo transcurrido desde el evento anterior.

Una vez finalizada una de esas 10 simulaciones, las métricas de rendimiento se calculan en la función *calculate_metrics(state)*, utilizando los acumuladores estadísticos actualizados durante la ejecución de los eventos de arribo y salida. Las métricas calculadas son:

Métrica	Fórmula
Utilización (ρ)	λ/μ
Clientes en sistema (L)	$\rho/(1-\rho)$
Clientes en cola (L_q)	$\rho^2/(1-\rho)$
Tiempo en sistema (W)	$1/(\mu-\lambda)$
Tiempo en cola (W_q)	$\lambda/[\mu(\mu-\lambda)]$
$P(n \text{ en cola})$	$(1-\rho)\cdot\rho^n$

En el código:

```
utilization = state['area_server_status'] / sim_time
```

Donde la variable *area_server_status* acumula el área bajo la curva del estado del servidor (1 cuando está ocupado y 0 cuando está libre).

```
L_q = state['area_num_in_q'] / sim_time
```

La variable *area_num_in_q* almacena el área bajo la curva del tamaño de la cola.

```
L = L_q + utilization
```

Se aprovecha que, en un sistema con un único servidor, la cantidad promedio de clientes en servicio coincide con la utilización del servidor.

```
W_q = state['total_of_delays'] / num_delayed
```

La variable *total_of_delays* acumula todos los retrasos sufridos por los clientes antes de iniciar servicio.

```
W = state['total_time_in_system'] / num_delayed
```

La variable *total_time_in_system* acumula, para cada cliente, la suma del tiempo de espera más su tiempo de servicio.

```
for n, time_spent in state['time_in_q_state'].items():
    prob_n_in_q[n] = time_spent / sim_time
```

Donde *prob_n_in_q* es la probabilidad de encontrar n clientes en cola. La estructura *time_in_q_state* almacena el tiempo acumulado que el sistema permaneció en cada estado de cola.

Probabilidad de denegación de servicio: Esta métrica es relevante cuando la cola tiene capacidad finita (K). Mide la proporción de clientes que fueron rechazados por encontrar la cola completa.

```
prob_blocking = blocked / arrived
```

Si la cola es infinita (modelo M/M/1), esta probabilidad es igual a cero porque ningún cliente puede ser rechazado.

Una vez terminadas las corridas, para graficar los resultados se utiliza la biblioteca matplotlib.

3.1.2. Modelo de Inventario

La simulación permite representar el comportamiento dinámico de un sistema sometido a demanda aleatoria, tiempos de entrega variables y diferentes componentes de costo.

Tiene tres componentes fundamentales de costo: costo de ordenamiento, costo de mantenimiento de inventario y costo por faltantes. El desempeño de cada política se evalúa a través de múltiples réplicas independientes, lo que permite obtener resultados estadísticamente más robustos y reducir la influencia de la variabilidad aleatoria inherente al sistema.

De manera simple, el código de este modelo se basa en lo siguiente:

- Se revisa el inventario cada cierto período
- Si el stock cae por debajo del Punto de Reorden → Se hace un pedido
- El pedido llega tras un tiempo (que puede ser fijo o aleatorio)

3.1.2.1 Definición de Parámetros El sistema inicia con un conjunto de parámetros que describen el comportamiento del inventario. Estos son:

- Inventario inicial (*INITIAL_INV_LEVEL*).
- Horizonte de simulación (*NUM_MONTHS*).
- Promedio de llegada de demandas (*MEAN_INTERDEMAND*).
- Tamaño de la demanda (*PROB_DISTRIB_DEMAND*): La demanda puede tomar valores discretos entre 1 y 4 unidades. La distribución acumulada utilizada genera probabilidades equivalentes a:

Demanda	Probabilidad
1	1/6
2	1/3
3	1/3
4	1/6

- Costo de ordenamiento (*SETUP_COST*): Representa el costo fijo asociado a realizar un pedido.
- Costo variable por unidad solicitada (*INCREMENTAL_COST*).
- Costo de mantenimiento (*HOLDING_COST*): Corresponde al costo de almacenar una unidad durante un mes.
- Costo de faltante (*SHORTAGE_COST*): Penaliza la falta de inventario cuando la demanda supera las existencias disponibles.

3.1.2.2 Generación de Variables Aleatorias El comportamiento estocástico del sistema se modela mediante dos distribuciones probabilísticas. Para cada período, es decir, por cada mes, se reciben los pedidos pendientes que llegan en ese mes y se genera una demanda aleatoria para ese mes.

Para generar tiempos entre demandas se utiliza una distribución exponencial:

```
random.expovariate(1.0 / mean)
```

Esta distribución es adecuada para representar procesos de llegadas aleatorias independientes.

Para generar tiempos de entrega de pedidos se utiliza una distribución Uniforme:

```
random.uniform(a,b)
```

Donde el tiempo de entrega puede variar entre 0.5 y 1.0 meses:

```
MINLAG = 0.5
MAXLAG = 1.0
```

3.1.2.3 Métricas a evaluar Al finalizar cada réplica, el programa calcula una serie de métricas que permiten cuantificar los costos asociados a la gestión del inventario y comparar objetivamente las diferentes políticas (*s,S*).

Dado que el modelo es estocástico, una sola ejecución no es suficiente para obtener conclusiones confiables. Por ello se realizan diez réplicas independientes para cada política. Para cada métrica se calculan el promedio, que representa el comportamiento esperado de la política, y el desvío, que mide la dispersión de los resultados obtenidos entre las distintas réplicas.

```
statistics.mean(values)
statistics.stdev(values)
```

A través de los costos de ordenamiento, mantenimiento y faltantes, el modelo cuantifica los distintos compromisos asociados a la gestión del stock. La combinación de estos componentes en el costo total promedio, junto con el análisis de la variabilidad entre réplicas, permite identificar la política (s,S) más eficiente y económicamente conveniente para el sistema estudiado.

3.2. AnyLogic

Para la implementación de los modelos en AnyLogic, se adoptó un enfoque de modelado centrado en procesos (*Process-Centric Modeling*). Este paradigma permite representar la dinámica del sistema mediante un flujo lógico de entidades a través de una red de bloques predefinidos, lo que facilita el seguimiento de estados y la recolección de estadísticas en tiempo real. [1]

3.2.1. Implementación del Modelo M/M/1 y M/M/1/K

El sistema de colas se construyó usando la *Process Modeling Library* (PML), traduciendo los componentes teóricos de la fila de espera en bloques funcionales. Estructura del modelo: [1]

- **Generación de arribos:** Se usó un bloque *Source*, configurado para generar entidades siguiendo una distribución exponencial con tasa λ . [2]
- **Gestión de cola:** Se usó un bloque *Queue*. Para los experimentos de cola finita (M/M/1/K), se usó la propiedad *Maximum capacity* con los valores requeridos (0, 2, 5, 10 y 50). Las entidades rechazadas por falta de espacio se derivaron a un bloque *Sink* secundario para calcular la probabilidad de denegación de servicio. [2]
- **Mecanismo de servicio:** Se usó un bloque *Delay* para representar el tiempo de atención, parametrizado con una distribución exponencial de tasa ω . [2]
- **Captura de datos:** Se usaron bloques *TimeMeasureStart* y *TimeMeasureEnd* para registrar el tiempo promedio en el sistema (W) y en cola (Wq). Para las medidas de longitud (L, Lq), se usaron objetos *Statistics* de la paleta de análisis, capturando de forma continua el número de entidades presentes en los bloques respectivos. [3]

3.2.2. Implementación del Modelo de Inventario

El modelo de inventario se desarrolló combinando bloques de procesos con lógica de eventos cíclicos para representar la política estacionaria (s, S). [3]

Se definió una variable de estado para el nivel de inventario físico (I). Un evento programado hace una revisión periódica al inicio de cada mes. Si $I < s$, se genera una entidad “orden” que atraviesa un bloque *Delay* que representa el retraso de entrega (*delivery lag*) estocástico. Al terminar este retraso, la entidad actualiza la variable de stock. Los costos de mantenimiento y faltante se calcularon por el *promedio temporal* del inventario positivo y negativo, respectivamente, usando la capacidad de integración estadística de AnyLogic. [4]

3.2.3. IU y experimentación

Se construyó una interfaz en la ventana *Main* que incluye controles tipo *Slider* y campos de entrada de datos. Estos permiten cambiar las tasas de arribo y los puntos de reorden en tiempo real, actualizando instantáneamente los gráficos de salida (histogramas y diagramas de dispersión). [5]

Se configuraron experimentos de simulación para ejecutar un mínimo de 10 corridas por escenario, asegurando que los resultados representen la convergencia de los estadísticos y no una fluctuación aleatoria aislada. [5]

3.3. Parámetros elegidos para el modelo de Inventario

Duración de la simulación (NUM_MONTHS): se estableció una duración de 120 meses (10 años). Esta longitud es suficiente para que las estimaciones de costo converjan y disminuya la varianza entre réplicas.

Tiempo promedio entre demandas (MEAN_INTERDEMAND): se usó un valor de 0,1 meses, que es igual a 10 pedidos por mes aprox. Este es el valor estándar que se usa en el modelo clásico de Law & Kelton.

Tamaño de la demanda (PROB_DISTRIB_DEMAND): se configuró una distribución discreta simétrica para demandas entre 1 y 4 unidades (con probabilidades 1/6, 1/3, 1/3 y 1/6). Esto resulta en una media de 2,5 unidades por pedido aprox, representando la variabilidad típica de una demanda intermitente.

Retraso de entrega (MINLAG y MAXLAG): se definió un retraso uniforme entre 0,5 y 1 meses. Esto permite simular proveedores con tiempos de entrega moderadamente variables, donde los pedidos pueden llegar en el mismo mes de la orden o el mes siguiente.

Políticas de pedido (s, S): se eligieron 9 políticas que cubren combinaciones de puntos de reorden bajos, medios y altos con distintos niveles objetivo. Esta variedad permite analizar la relación y el equilibrio (frontera de Pareto) entre el costo de ordenamiento y el costo de faltante.

Número de réplicas (NUM_REPLICATIONS): se hicieron mínimo de 10 corridas independientes por cada experimento. Esta cantidad es fundamental para poder calcular intervalos de confianza del 95 % y ver la variabilidad estocástica inherente al modelo.

Para el modelo MM1, los parámetros de tasa de arribo (λ) se variaron al 25 %, 50 %, 75 %, 100 % y 125 % respecto a la tasa de servicio (ω).

4. Resultados

4.1. Análisis del Modelo M/M/1

La salida de una corrida de nuestro programa.

```
=====
REPORTE DE SIMULACIÓN - MODELO DE COLA M/M/1
=====
Parámetros de Entrada:
Tiempo promedio entre arribos (1/lambda): 1.000 minutos
Tiempo promedio de servicio (1/mu):      0.800 minutos
Clientes requeridos en simulación:      10000
Límite de la cola (K):                   Infinito (M/M/1)
-----
Medidas de Rendimiento:
Tiempo de simulación transcurrido:      10044.780 minutos
Clientes que iniciaron servicio:        10000
Clientes arribados en total:            10005
Clientes bloqueados:                    0
-----
1. Promedio de clientes en el sistema (L): 3.9742
2. Promedio de clientes en cola (Lq):     3.1722
3. Tiempo promedio en el sistema (W):    3.9917 minutos
4. Tiempo promedio en cola (Wq):         3.1861 minutos
5. Utilización del servidor (Rho):       0.8020
6. Probabilidad de denegación de servicio: 0.0000 (0.00%)
-----
```

4.1.1. Evolución Temporal y Estabilización

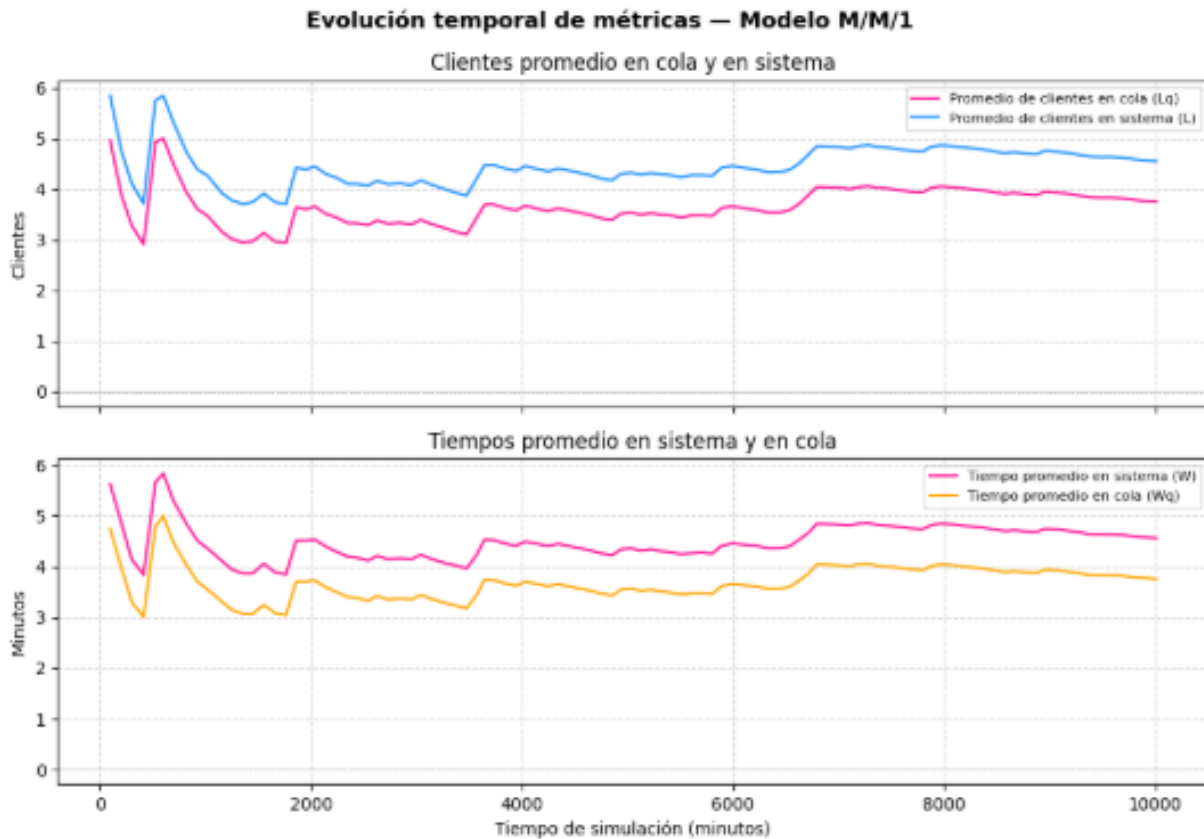


Figura 1: Evolución temporal de métricas - Modelo MM1

Se generaron gráficos de evolución temporal, usando la función `plot_metrics` de Python, para ver la convergencia de los estadísticos. En los gráficos se distinguen 2 fases:

- **Periodo transitorio:** al inicio de la simulación (tiempos bajos), se ven fluctuaciones significativas en el número promedio de clientes (L , L_q) y los tiempos de espera (W , W_q).
- **Estado estacionario:** a medida que el tiempo de la simulación avanza y se procesa el número requerido de clientes ($NUM_DELAYS_REQUIRED = 10000$), las curvas de las métricas tienden a estabilizarse en valores constantes. Esta convergencia confirma que el sistema alcanzó el estado estable, permitiendo que los resultados de la simulación sean comparables con los valores teóricos calculados con las fórmulas de Little ($L = \lambda W$).

Time Plots (Evolución Temporal y Estabilización) - AnyLogic



Figura 2: Evolución temporal y estabilización - AnyLogic

En estos gráficos, se ve la evolución de las métricas L (clientes) y W (tiempo) en AnyLogic. Permite distinguir el período transitorio inicial (0-20 unidades de tiempo) y la posterior convergencia hacia el estado estacionario. La estabilidad que se ve en los valores promedios ($L=0,5$, $W=1$) valida la capacidad del modelo para representar el comportamiento de largo plazo del sistema.

Scatter Plot (Sensibilidad ante la tasa de arribo) - AnyLogic

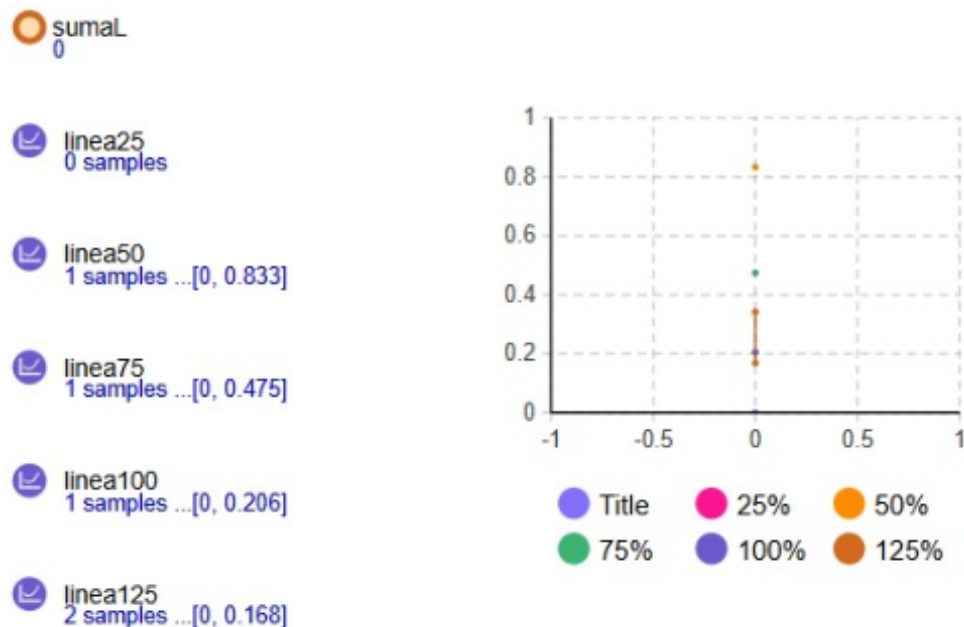


Figura 3: Sensibilidad ante la tasa de arribo - AnyLogic

Para cumplir con el requerimiento de variar la tasa de arribo entre el 25 % y 125 % de la tasa de servicio, se fueron variando los parámetros en AnyLogic. En el gráfico se ve la recolección de muestras para cada escenario. Este análisis

permite comparar cómo el sistema responde al aumento de carga, viendo la transición de un sistema fluido (25 %-75 %) hacia uno saturado o inestable (100 %-125 %) en el caso de colas infinitas.

4.1.2. Sensibilidad ante la tasa de arribo (λ)

Se hicieron experimentos variando la tasa de arribo al 25 %, 50 %, 75 %, 100 % y 125 % respecto a la tasa de servicio (ω).

Primero se utilizaron los siguientes valores, donde la tasa de servicio corresponde a un 25 % de la tasa de arribo:

```
MEAN_INTERARRIVAL = 12 # Tiempo promedio entre arribos (minutos)
MEAN_SERVICE = 3 # Tiempo promedio de servicio (minutos)
NUM_DELAYS_REQUIRED = 10000 # Cantidad de clientes a procesar
# (que completan retraso e inician servicio)
QUEUE_LIMIT = float('inf') # Límite de la cola (cantidad de clientes
# que pueden estar en cola) M/M/1/K - float('inf') para M/M/1
```

Que dieron el siguiente resultado en una corrida:

```
=====
REPORTE DE SIMULACIÓN - MODELO DE COLA M/M/1
=====
Parámetros de Entrada:
Tiempo promedio entre arribos (1/lambda): 12.000 minutos
Tiempo promedio de servicio (1/mu):      3.000 minutos
Clientes requeridos en simulación:      10000
Límite de la cola (K):                    Infinito (M/M/1)
-----
1. Promedio de clientes en el sistema (L): 0.3226
2. Promedio de clientes en cola (Lq):      0.0762
3. Tiempo promedio en el sistema (W):      3.8939 minutos
4. Tiempo promedio en cola (Wq):           0.9197 minutos
5. Utilización del servidor (Rho):         0.2464
-----
7. Probabilidad de encontrar n clientes en cola (P(Nq = n)):
n = 0: Probabilidad = 0.9415 (94.15%)
n = 1: Probabilidad = 0.0451 ( 4.51%)
n = 2: Probabilidad = 0.0102 ( 1.02%)
n = 3: Probabilidad = 0.0025 ( 0.25%)
n = 4: Probabilidad = 0.0006 ( 0.06%)
n = 5: Probabilidad = 0.0001 ( 0.01%)
n = 6: Probabilidad = 0.0000 ( 0.00%)
n = 7: Probabilidad = 0.0000 ( 0.00%)
=====
```

Luego se utilizaron los siguientes valores, donde la tasa de servicio corresponde a un 50 % y un 75 % respectivamente de la tasa de arribo:

```
MEAN_INTERARRIVAL = 6 # Tiempo promedio entre arribos (minutos)
MEAN_SERVICE = 3 # Tiempo promedio de servicio (minutos)
NUM_DELAYS_REQUIRED = 10000 # Cantidad de clientes a procesar
# (que completan retraso e inician servicio)
QUEUE_LIMIT = float('inf') # Límite de la cola (cantidad de clientes
# que pueden estar en cola) M/M/1/K - float('inf') para M/M/1

MEAN_INTERARRIVAL = 4 # Tiempo promedio entre arribos (minutos)
MEAN_SERVICE = 3 # Tiempo promedio de servicio (minutos)
NUM_DELAYS_REQUIRED = 10000 # Cantidad de clientes a procesar
# (que completan retraso e inician servicio)
QUEUE_LIMIT = float('inf') # Límite de la cola (cantidad de clientes
# que pueden estar en cola) M/M/1/K - float('inf') para M/M/1
```

Que dieron los siguientes resultado en una corrida:

```
=====
REPORTE DE SIMULACIÓN - MODELO DE COLA M/M/1
=====
Parámetros de Entrada:
Tiempo promedio entre arribos (1/lambda): 6.000 minutos
Tiempo promedio de servicio (1/mu):      3.000 minutos
Clientes requeridos en simulación:      10000
Límite de la cola (K):                   Infinito (M/M/1)
-----
Medidas de Rendimiento:
Tiempo de simulación transcurrido:      59335.333 minutos
-----
1. Promedio de clientes en el sistema (L): 1.0314
2. Promedio de clientes en cola (Lq):     0.5235
3. Tiempo promedio en el sistema (W):     6.1196 minutos
4. Tiempo promedio en cola (Wq):          3.1061 minutos
5. Utilización del servidor (Rho):        0.5079
-----
7. Probabilidad de encontrar n clientes en cola (P(Nq = n)):
n = 0: Probabilidad = 0.7406 (74.06%)
n = 1: Probabilidad = 0.1288 (12.88%)
n = 2: Probabilidad = 0.0650 ( 6.50%)
n = 3: Probabilidad = 0.0321 ( 3.21%)
n = 4: Probabilidad = 0.0162 ( 1.62%)
n = 5: Probabilidad = 0.0097 ( 0.97%)
n = 6: Probabilidad = 0.0039 ( 0.39%)
n = 7: Probabilidad = 0.0014 ( 0.14%)
n = 8: Probabilidad = 0.0009 ( 0.09%)
n = 9: Probabilidad = 0.0006 ( 0.06%)
n = 10: Probabilidad = 0.0004 ( 0.04%)
n = 11: Probabilidad = 0.0004 ( 0.04%)
n = 12: Probabilidad = 0.0001 ( 0.01%)
n = 13: Probabilidad = 0.0000 ( 0.00%)
=====

=====
REPORTE DE SIMULACIÓN - MODELO DE COLA M/M/1
=====
Parámetros de Entrada:
Tiempo promedio entre arribos (1/lambda): 4.000 minutos
Tiempo promedio de servicio (1/mu):      3.000 minutos
Clientes requeridos en simulación:      10000
Límite de la cola (K):                   Infinito (M/M/1)
-----
Medidas de Rendimiento:
Tiempo de simulación transcurrido:      40028.803 minutos
-----
1. Promedio de clientes en el sistema (L): 3.0077
2. Promedio de clientes en cola (Lq):     2.2604
3. Tiempo promedio en el sistema (W):     12.0394 minutos
4. Tiempo promedio en cola (Wq):          9.0477 minutos
5. Utilización del servidor (Rho):        0.7473
-----
7. Probabilidad de encontrar n clientes en cola (P(Nq = n)):
n = 0: Probabilidad = 0.4436 (44.36%)
n = 1: Probabilidad = 0.1368 (13.68%)
n = 2: Probabilidad = 0.0958 ( 9.58%)
n = 3: Probabilidad = 0.0742 ( 7.42%)
```

n = 4: Probabilidad = 0.0625 (6.25%)
 n = 5: Probabilidad = 0.0488 (4.88%)
 n = 6: Probabilidad = 0.0363 (3.63%)
 n = 7: Probabilidad = 0.0265 (2.65%)
 n = 8: Probabilidad = 0.0197 (1.97%)
 n = 9: Probabilidad = 0.0156 (1.56%)
 n = 10: Probabilidad = 0.0118 (1.18%)
 n = 11: Probabilidad = 0.0062 (0.62%)
 n = 12: Probabilidad = 0.0053 (0.53%)
 n = 13: Probabilidad = 0.0039 (0.39%)
 n = 14: Probabilidad = 0.0031 (0.31%)

Para casos donde $\rho < 1$ (25 % a 75 %), el sistema es estable y los tiempos de espera crecen de forma proporcional a la carga. En el 100 % ($\lambda = \omega$), se ve una tendencia hacia el infinito en la longitud de la cola en sistemas MM1 infinitos. Pero en la simulación el tiempo finito de ejecución muestra una cola que crece continuamente sin estabilizarse, confirmando la **inestabilidad** del modelo bajo saturación total.

Para un 100 % ($\lambda = \omega$):

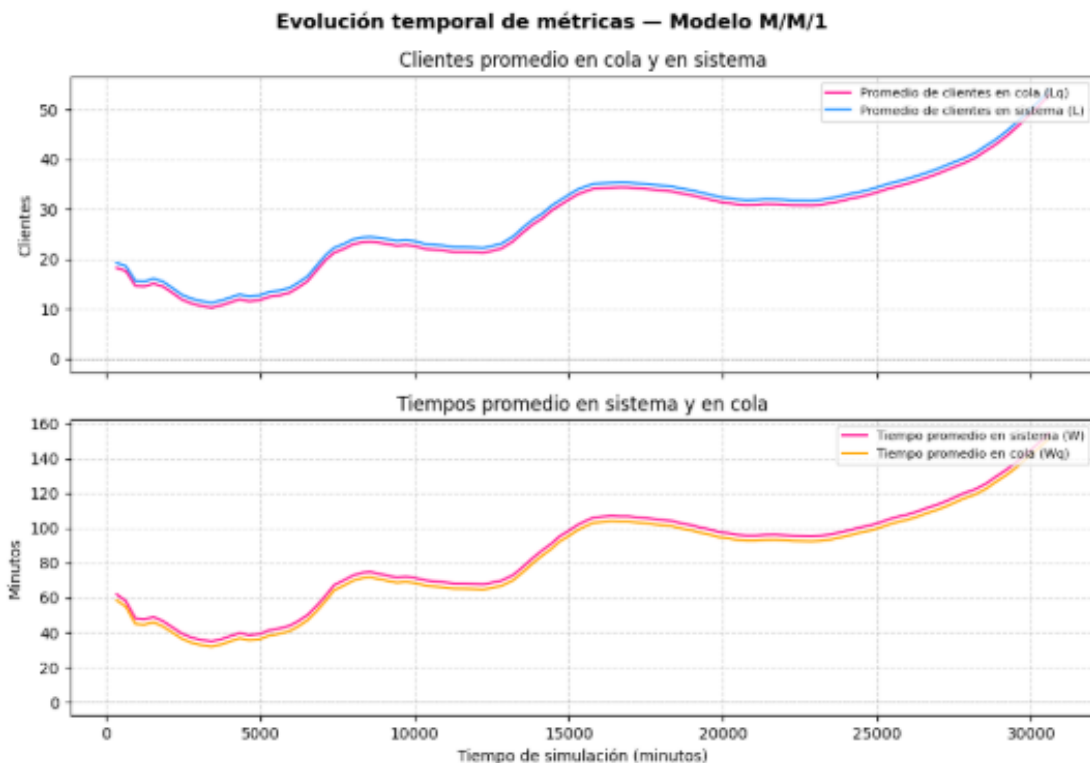


Figura 4: Evolución temporal de métricas - Modelo MM1

=====

REPORTE DE SIMULACIÓN - MODELO DE COLA M/M/1

=====

Parámetros de Entrada:

Tiempo promedio entre arribos ($1/\lambda$): 3.000 minutos
 Tiempo promedio de servicio ($1/\mu$): 3.000 minutos
 Clientes requeridos en simulación: 10000
 Límite de la cola (K): Infinito (M/M/1)

Medidas de Rendimiento:

Tiempo de simulación transcurrido: 30585.481 minutos

 1. Promedio de clientes en el sistema (L): 53.8815
 2. Promedio de clientes en cola (Lq): 52.8916
 3. Tiempo promedio en el sistema (W): 154.4522 minutos
 4. Tiempo promedio en cola (Wq): 151.4238 minutos
 5. Utilización del servidor (Rho): 0.9899
 6. Probabilidad de denegación de servicio: 0.0000 (0.00%)

 7. Probabilidad de encontrar n clientes en cola ($P(N_q = n)$):
 n = 0: Probabilidad = 0.0200 (2.00%)
 n = 1: Probabilidad = 0.0093 (0.93%)
 n = 2: Probabilidad = 0.0095 (0.95%)
 n = 3: Probabilidad = 0.0103 (1.03%)
 n = 4: Probabilidad = 0.0102 (1.02%)
 n = 5: Probabilidad = 0.0124 (1.24%)

Donde vemos una clara distorsión de las salidas, afirmando que el sistema es inestable.

4.1.3. Análisis de cola finita (M/M/1/K) y Denegación de servicio

Para la cola finita ($K \in \{0, 2, 5, 10, 50\}$), se evaluó el impacto del límite de capacidad sobre la probabilidad de denegación de servicio.

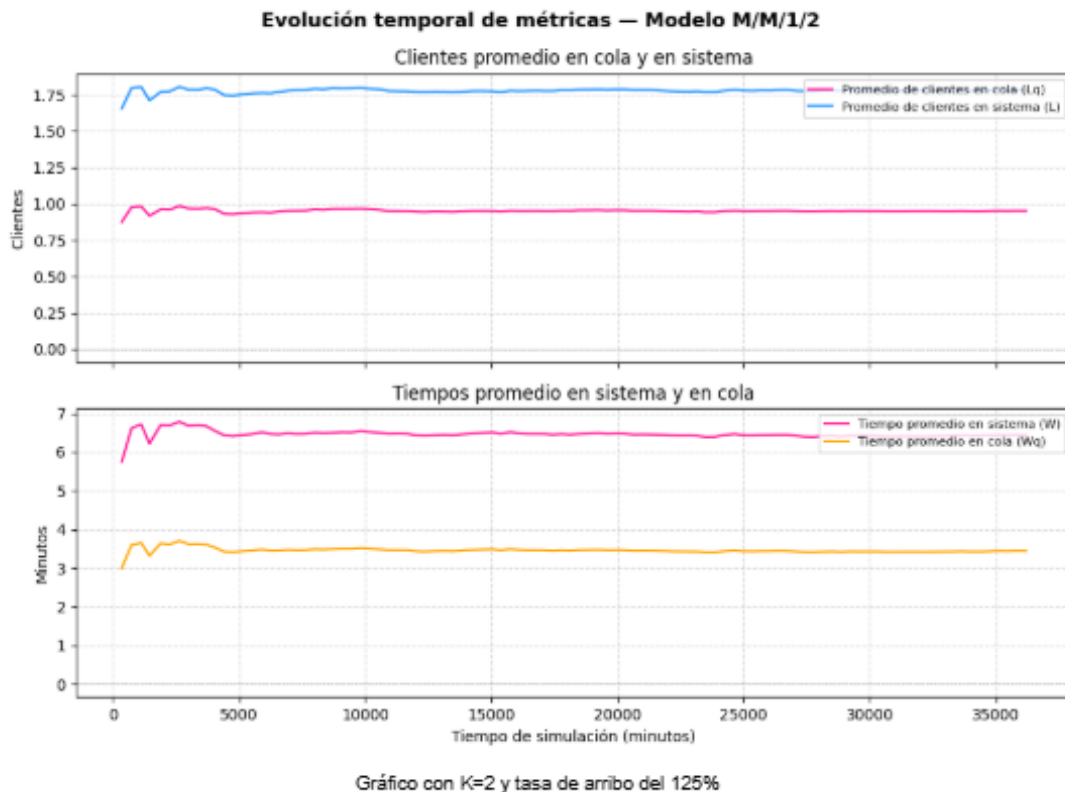


Figura 5: Evolución temporal de métricas - Modelo M/M/1/2

TABLA COMPARATIVA: EFECTO DEL TAMAÑO DE COLA (K) SOBRE LA DENEGACIÓN DE SERVICIO

Cola (K)	Arribos	Bloqueados	P(Denegación)	Utilización	Lq
0	18172	8172	0.449703	0.4456	0.0000
2	12140	2140	0.176277	0.6536	0.5597
5	10766	766	0.071150	0.7575	1.4841
10	10267	259	0.025226	0.7944	2.5609
50	10005	0	0.000000	0.8020	3.1722

Los resultados (que se ven en la tabla generada por el script de Python) muestran que a menor tamaño de cola (K) mayor cantidad de clientes bloqueados (`num_custs_blocked`).

A diferencia del modelo infinito, incluso con tasas de arribo del 125 % ($\rho > 1$), los modelos MM1K alcanzan un estado estable. Esto pasa porque el mecanismo de rechazo de clientes limita el uso real del servidor y evita el crecimiento indefinido de la cola de espera.

4.1.4. Probabilidad de encontrar n clientes en cola

Usando `time_in_q_state` en el código Python, se calculó la fracción de tiempo que el sistema permaneció con exactamente n clientes esperando. Los resultados muestran que para un sistema estable, la probabilidad disminuye exponencialmente cuando n aumenta, siguiendo la distribución teórica $P_n = (1 - \rho)\rho^n$ para el sistema total.

4.2. Análisis del Modelo de Inventario

Se hizo una simulación con una duración de 120 meses. Para cada una de las 9 políticas elegidas (que varían desde un punto de reorden $s = 20$ hasta $s = 60$, y niveles de inventario objetivo S de hasta 100 unidades), se ejecutaron 10 réplicas independientes usando semillas aleatorias distintas. Esto permite calcular el promedio de los costos y su desvío estándar.

Con la función `calculate_metrics(state)` de Python, se analizaron los componentes que integran el Costo Total Promedio Mensual:

- **Costo de ordenamiento (*avg_ordering_cost*):** representa el gasto por hacer pedidos al proveedor. Incluye un costo fijo de preparación (`SETUP_COST = 32,0`) y un costo variable por unidad (`INCREMENTAL_COST = 3,0`). Al aumentar el valor de S para un mismo s , la frecuencia de los pedidos disminuye, y disminuye el *avg_ordering_cost*.
- **Costo de mantenimiento (*avg_holding_cost*):** Se calcula multiplicando el costo unitario (`HOLDING_COST = 1,0`) por el promedio temporal del inventario positivo (*area_holding*). Las políticas con valores de s y S más altos mantienen niveles de stock físico más alto.
- **Costo de faltante (*avg_shortage_cost*):** Penaliza el inventario negativo o backlog (`SHORTAGE_COST = 5,0`). Al aumentar el punto de reorden s , disminuye el área de faltantes, asegurando que el pedido se haga antes de agotar las existencias.

El script genera una tabla resumen comparativa de políticas que muestra los promedios de las 10 réplicas para cada configuración. El criterio de selección usado es la minimización del Costo Total Promedio Mensual (*total_cost_mean* en Python y *datosCostoTotal* en AnyLogic).

La comparación de los resultados permite ver cuál de las 9 combinaciones de (s, S) es la política más eficiente para las condiciones de demanda (`MEAN_INTERDEMAND = 0,1` y `NUM_VALUES_DEMAND = 4`) y retraso de entrega analizadas (`MINLAG = 0,5` y `MAXLAG = 1,0`). Esta política va a ser la que logra el costo total más bajo.

Time Plot (Nivel de Inventario) - AnyLogic Este gráfico representa la evolución dinámica del stock físico en el depósito durante una corrida individual de 120 meses. Se ve como el nivel de inventario (*inventoryLevel*) desciende de forma escalonada por la llegada de demandas discretas (entre 1 y 4 unidades) que siguen un proceso de Poisson con una tasa de 10 pedidos por mes. Cada vez que la curva cae por debajo del punto de reorden $s = 20$, el evento mensual de revisión (*inventoryReview*) dispara una orden de compra hacia el bloque *orderSource*.

La gráfica muestra tramos donde el inventario entra en valores negativos (alrededor de -20). Esto pasa durante el retraso de entrega (lead time) de entre 0,5 y 1 meses, representando el backlog o pedidos pendientes que generan el costo de faltante (*statsNegativeInv*). Al terminar el retraso en el bloque *deliveryDelay*, el stock salta instantáneamente hasta alcanzar el nivel objetivo $S = 60$, reiniciando el ciclo.



Figura 6: Nivel de Inventario - AnyLogic

Histograma de Costo Total - AnyLogic El histograma muestra los resultados de 10 réplicas independientes (“10 samples”). El costo total promedio obtenido es de 116,494 (*Mean*). Este valor integra el costo de ordenamiento promedio, el de mantenimiento y el de faltante, calculados mediante promedios temporales continuos (áreas bajo la curva).

La dispersión de las barras entre los valores de 112,5 y 122,1 muestra como la aleatoriedad en el tamaño de la demanda y en los tiempos de entrega afecta el costo operativo mensual. La concentración de las muestras alrededor de la media valida la robustez del modelo y la convergencia al estado estable tras el periodo transitorio inicial.

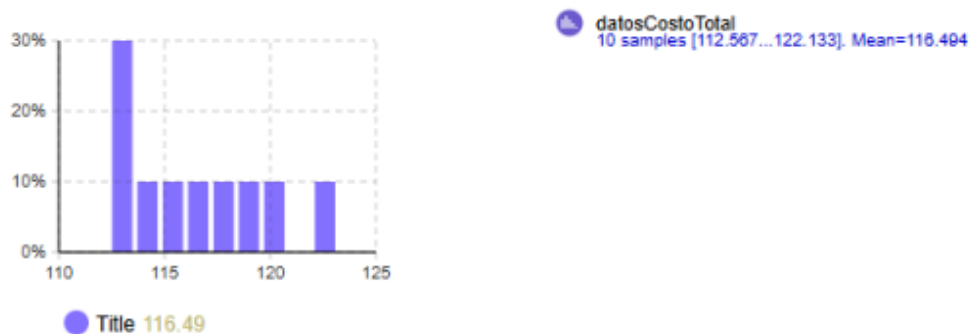


Figura 7: Costo Total - AnyLogic

4.2.1. Comparativa entre valor teórico, Python y AnyLogic

Política (s,S)	Valor teórico	Python	AnyLogic
(20, 40)	126,61	124,6153	116,283
(20, 60)	122,74	117,9460	116,494
(20, 80)	123,86	121,5325	121,402
(20, 100)	125,32	127,3081	128,039
(40, 60)	126,37	125,6530	130,804
(40, 80)	125,46	124,6290	131,718
(40, 100)	132,34	131,1686	136,664
(60, 80)	150,02	143,5591	152,275
(60, 100)	143,20	143,8565	152,293

5. Discusión

El desarrollo de este trabajo práctico permitió validar la importancia de la simulación computacional para analizar sistemas estocásticos, contrastando los resultados teóricos matemáticos con implementaciones prácticas en Python y modelos visuales en AnyLogic. A partir de la recolección y análisis de las métricas de rendimiento, se destacan los siguientes puntos fundamentales:

5.1. Comportamiento del Sistema de Colas y la Condición de Estabilidad

En el análisis del modelo M/M/1, los experimentos demostraron empíricamente que cuando el sistema opera bajo una condición de estabilidad (con tasas de arribo variando entre el 25 % y el 75 % respecto a la de servicio), los tiempos de espera crecen de forma proporcional a la carga y las métricas logran converger hacia valores constantes. Sin embargo, el fenómeno más destacable ocurrió al forzar el sistema al límite de su capacidad (100 %, donde $\lambda = \mu$). En este escenario, la simulación confirmó la inestabilidad inherente de una cola infinita: la longitud de la cola creció de forma continua sin llegar nunca a estabilizarse, generando una clara distorsión en las salidas del sistema. [1]

5.2. Contraste Analítico frente a la Simulación (Teoría vs. Python vs. AnyLogic)

Al observar la tabla comparativa final de costos de inventario, se verifica una notable coherencia general entre las tres fuentes de datos evaluadas. Las ligeras discrepancias numéricas detectadas —por ejemplo, para la política (20, 40), donde el valor matemático teórico es de 126,61, mientras que Python arrojó 124,6153 y AnyLogic 116,283— son completamente esperables. **Estas diferencias no indican errores de modelado**, sino que son el reflejo directo de la naturaleza del muestreo estocástico. A diferencia de los modelos teóricos que calculan expectativas matemáticas cerradas para un tiempo infinito, tanto Python como AnyLogic evalúan una muestra temporal finita (120 meses) que está inevitablemente afectada por el azar de la demanda y el lead time. En conclusión, la simulación probó ser una herramienta superior para apreciar no solo el costo esperado, sino también la variabilidad y el riesgo que se presentan en el mundo real. [2]

6. Conclusiones

Comparando el valor teórico esperado, la programación en Python y el modelo en AnyLogic, se tienen las siguientes conclusiones:

Tanto los scripts de Python como los flujos de procesos en AnyLogic son coherentes con los modelos matemáticos de Law y Kelton. Las diferencias ligeras que hay son intrínsecas a la naturaleza aleatoria de los sistemas y al uso de una duración finita (120 meses), a diferencia de las fórmulas teóricas que asumen un estado estable ideal de tiempo infinito.

La condición de estabilidad ($\rho < 1$) es el factor determinante en el rendimiento de las colas infinitas. Se observó cómo al alcanzar la saturación (100 %), el sistema se vuelve inestable, mientras que la implementación de colas finitas (M/M/1/K) actúa como regulador y estabiliza el sistema a costa de la denegación de servicio.

El análisis de las 9 políticas (s, S) demostró que la simulación permite identificar el punto de equilibrio óptimo entre los costos de ordenamiento, mantenimiento y faltante. La política (20, 60) es la más eficiente para las condiciones dadas, logrando minimizar el Costo Total Promedio Mensual frente a la variabilidad de la demanda y el lead time del proveedor.

La ejecución de mínimo 10 replicaciones independientes por experimento se hizo para disminuir el impacto de la variabilidad estocástica. Usar histogramas y diagramas de dispersión permitió confirmar que los promedios obtenidos no son fluctuaciones aisladas, sino estimaciones robustas que tienden a la normalidad según el Teorema Central del Límite.

En definitiva, la integración de estas herramientas permite obtener resultados numéricos precisos, comprender el comportamiento dinámico y los riesgos de los sistemas reales antes de su implementación física, facilitando tomar decisiones basadas en evidencia estadística.

Referencias

[1] Averill M. Law. Simulation modeling and analysis, 2014.